

I found some (active) students went into hibernation from the time the lecture started today. I have written the following note as it could help those, and to all others, as well.

Computation History and PDA Construction

Consider the set

$V = \{x \mid x \text{ is a valid computation history of TM } M \text{ on } w \text{ with } x \text{ in the form } x_1 \# x_2^R \# x_3 \# x_4^R \# \dots \# x_m \text{ where every } x_i \text{ is a string}\}.$

We constructed two PDAs P_1 and P_2 such that $L(P_1) \cap L(P_2) = V$, where

$$\begin{aligned} L(P_1) &= (x_i \# x_{i+1}^R \#)^* (\{\epsilon\} \cup \Gamma^* q_f \Gamma^* \#), \\ L(P_2) &= q_0 \Sigma^* \# (x_i^R \# x_{i+1} \#)^* (\{\epsilon\} \cup \Gamma^* q_f \Gamma^* \#), \end{aligned}$$

where x_i yields x_{i+1} on M .

Consider the language $L(P_1)$. Since union, concatenation, and Kleene star are closed for regular languages, leaving aside $x_i \# x_{i+1}^R$, the rest is a regular language. As discussed in class, we take the help of a stack to ensure x_i yields z and z^R is indeed equal to x_{i+1}^R . In doing this, we assumed M is a DTM. Since we need a stack for a submodule, and since union, concatenation, and Kleene star are closed for CFLs, we will have a PDA to precisely accept $L(P_1)$. Hence, $L(P_1)$ is a CFL, and has a CFG G_1 such that $L(G_1) = L(P_1)$.

Significantly, as the input x_1 is read, P_1 builds a stack to have z from bottom-to-top; while traversing x_2 , P_1 determines whether x_2^R is the same as z^R ; then, P_1 reads x_3 and builds a stack to z' , etc.

The reasoning is similar for $L(P_2)$ being a CFL. And, as the input x_1 is read, since we want to force P_2 to check x_i yields x_{i+1} for even i , we let P_2 make sure x_1 is of the form $q_0 \Sigma^*$. Upon reading x_2 , P_2 pushes symbols onto the stack to ensure x_2 yields x_3 , etc.

If m is even, x_{m-1} yields x_m is ensured by P_1 . In that case, P_1 , by using the ϵ option, will not check whether x_m is of the form $\Gamma^* q_f \Gamma^*$. However, in this case, P_2 's job is over by the time it completes reading x_{m-1} , and hence, P_2 checks whether x_m is in the right form by checking the state symbol in the last substring (x_m) it traversed is q_f upon finding a blank symbol. This analysis is symmetric if m is odd.

Since context-free languages are not closed under intersection, we could not merge P_1 and P_2 into one PDA. Hence, concluded, $V = L(P_1) \cap L(P_2)$.

Invalid Computation History

Consider the set $V' = \{x \mid x \text{ is an invalid computation history of TM } M \text{ on } w \text{ with } x \text{ in the form } \#x_1 \#x_2^R \#x_3 \#x_4^R \# \dots \#x_m \text{ where every } x_i \text{ is a string}\}.$

With PDA P'_1 , we non-deterministically choose a $\#$ in x . If i is odd for the x_i following that $\#$, then P'_1 checks x_i yields x_{i+1} ; if not, P'_1 accepts x ; if i is

even, P'_1 rejects. Analogously, with PDA P'_2 , we non-deterministically choose a $\#$ in x . If i is even for the x_i following that $\#$, then P'_2 checks x_i yields x_{i+1} ; if not, P'_2 accepts x ; if i is odd, P'_2 rejects.

The rest of the checks can be done with regular expressions and hence captured by a regular language L'' . Since the union of $L(P'_1)$, $L(P'_2)$, L'' is closed for CFLs, a PDA P' is designed such that $L(P') = V'$.

In designing P' , we accept x whenever any subpart is found to be not valid. Upon accepting x , all other nondeterministic threads get killed, meaning P' found an invalid portion in x . However, nondeterminism is not helpful in the case of designing a PDA for V since we need to ensure all subparts of x are valid (as described) before accepting x . Note that if any thread in a PDA accepts x , other threads get killed, and hence, it is not correct to accept x if x_i yields x_{i+1} for some i .